

kvm-tomcat-lab: an illustrated study guide

- [1. The story of what we built](#)
 - [Imagine a restaurant](#)
 - [What a big company version looks like](#)
 - [What happens when someone visits the page](#)
 - [Why bother with two rooms?](#)
 - [The order we built it in, and why that order](#)
 - [The numbers we ended up with](#)
 - [How to read the rest of this guide](#)
- [2. Machines inside machines, and the wires between them](#)
 - [A computer inside a computer \(virtualization\)](#)
 - [KVM, qemu, and libvirt - three names for one job](#)
 - [Why we ran Linux inside Windows \(WSL2\)](#)
 - [A ready-made computer: the cloud image](#)
 - [cloud-init: the note you leave for a new computer](#)
 - [Thick disks that are secretly thin \(qcow2 and backing files\)](#)
 - [The ceiling you set when you build the machine](#)
 - [The wires: a private network just for the machines](#)
 - [Getting in without a password \(SSH keys\)](#)
- [3. The kitchen, the waiter, and the bouncer](#)
 - [Java: the language the kitchen speaks](#)
 - [Tomcat: the kitchen itself](#)
 - [JSP: a web page with code inside it](#)
 - [WAR: the recipe box](#)
 - [Apache: the waiter at the door](#)
 - [reverse proxy: carrying the order to the kitchen](#)
 - [The bouncer: a firewall decides who may knock](#)
- [4. Doing it automatically, and protecting it](#)
 - [Why not just type commands like normal?](#)
 - [Ansible: the manager with a checklist](#)
 - [Idempotent: the most important idea in the project](#)
 - [Git and GitHub: a notebook that remembers everything](#)
 - [systemd timers: the alarm clock](#)
 - [tar and gzip: the box](#)
 - [SHA-256: the fingerprint](#)
 - [A manifest: the list of fingerprints](#)

- [The backup that checks itself](#)
- [rsync: the careful mover](#)
- [Snapshot versus backup: the photograph and the copy](#)
- [Growing a running machine \(live resize\)](#)
- [5. Watching, measuring, and finding the slow part](#)
 - [Why measure at all?](#)
 - [Prometheus: the nurse with the notebook](#)
 - [node_exporter: the little meter on each machine](#)
 - [A metric: a number with a timestamp](#)
 - [Alert rules: the rules that decide when to say something](#)
 - [Grafana: the chart on the wall](#)
 - [Docker: shipping containers for software](#)
 - [ab: pretending to be a crowd](#)
 - [The two numbers that matter: throughput and p95](#)
 - [What we found, and why the answer was interesting](#)
 - [Capacity planning: how many cooks do we need?](#)
- [6. Everything that broke, and what it teaches](#)
 - [Story 1: The bridge that would not come back](#)
 - [Story 2: The guard that could never say “no”](#)
 - [Story 3: It changed nothing, and it was lying](#)
 - [Story 4: The backup of nothing, which verified perfectly](#)
 - [Story 5: The command that printed nothing](#)
 - [Story 6: The machines kept rebooting themselves](#)
 - [Story 7: The processor arrived, but stayed asleep](#)
 - [Story 8: The clock that went backwards](#)
 - [The five lessons worth memorising](#)
- [7. Glossary, cheat sheet, and what to learn next](#)
 - [Glossary, in plain words](#)
 - [Command cheat sheet](#)
 - [Where you will meet all of this again](#)
 - [Sentences worth being able to say out loud](#)
 - [What to learn next, in a sensible order](#)
 - [The one-page summary](#)

1. The story of what we built

Imagine a restaurant

You built a small restaurant. Not a real one - a restaurant made of software, running inside your own laptop. Here is who works there:

In the restaurant	In our project	What it does
The building	Your laptop, with WSL2 and KVM	The place everything happens
Two locked rooms	<code>web01</code> and <code>app01</code>	Two separate computers inside one
The waiter at the door	Apache on <code>web01</code>	Greets every visitor and passes orders on
The kitchen	Tomcat on <code>app01</code>	Actually makes the food
The recipe	<code>labapp.war</code>	The instructions for the food
The manager with a checklist	Ansible	Walks around making sure everything matches the checklist
A photo you can rewind to	Snapshot	Lets you undo a mistake in seconds
A copy of the recipe at grandma's house	Backup	Saves you if the restaurant burns down
The nurse with a thermometer	Prometheus	Writes down numbers every 15 seconds
The chart on the wall	Grafana	Draws those numbers as pictures
The bouncer with the guest list	Firewall (<code>ufw</code>)	Decides who is allowed through which door

That is the whole project. Everything else is details about how those people do their jobs.

What a big company version looks like

Your restaurant seats 20 people. A company like a bank runs a restaurant with 5,000 tables, in three buildings, in two countries, with a second kitchen that takes over when the first one catches fire.

The workers are the same. There are just more of them, and more rules about what happens when something breaks. **That is why this project is worth building: the ideas are identical, only the size changes.**

What happens when someone visits the page

This is the single most useful thing to understand. Follow one visitor:

1. Someone opens a browser and asks for `http://192.168.122.12/labapp/health`.
2. `.12` is **web01**. The request arrives at Apache, the waiter.
3. Apache looks at its rule sheet (`labapp.conf`) which says: "anything asking for `/`, quietly pass it to `192.168.122.11:8080`". That is a **reverse proxy**: the visitor thinks they spoke to Apache, but Apache just carried the message to the kitchen.
4. `.11` is **app01**, and port `8080` is **Tomcat**, the kitchen.

5. Tomcat looks at the path `/labapp` and thinks “that is the app called `labapp`”, which it found in its `webapps` folder.
6. Inside `labapp` the file `WEB-INF/web.xml` says: the address `/health` belongs to the file `health.jsp`.
7. Tomcat runs `health.jsp`, which prints `OK app01` - the name of the machine that cooked it.
8. That text travels back the same way: Tomcat to Apache, Apache to the browser.

The visitor sees `OK app01`. Notice the name says **app01**, even though they asked **web01**. That is the proof the request really travelled between the two machines instead of being answered at the front door.

Analogy. You order soup. The waiter does not cook it, but they do hand it to you. If the bowl has the kitchen’s stamp on it, you know the kitchen really made it.

Why bother with two rooms?

A fair question. One computer could be both the waiter and the kitchen.

We split them because that is what real systems do, and each half gives you something:

- **One front door.** Visitors only ever see `web01`. The kitchen is hidden, so nobody can walk in and start poking the ovens. We proved this: after hardening, knocking directly on the kitchen door (`app01:8080`) from outside gets **no answer at all**.
- **One place to add security.** When you eventually add HTTPS, you do it once, at the front door. The kitchen never needs to know about certificates.
- **One place to write the guest book.** Every access log is in one file on `web01`, instead of scattered across machines.
- **Room to grow.** If the kitchen gets busy you can add a second kitchen later and have the waiter share orders between them. If the waiter gets busy, add a second waiter.
- **Smaller blast radius.** If the front door breaks, you can still reach the kitchen to fix it - and the other way round.

Analogy. A restaurant where customers walk straight into the kitchen is a food truck. Fine for one customer, chaos for a hundred.

The order we built it in, and why that order

The build happened in nine steps, called **modules**. The order is not random.

Step	What we did	Why it had to come in this position
1	Set up the machine that makes machines (KVM)	Nothing else exists until something can create a computer
2	Made two computers with fixed addresses	Ansible needs to know where they are, by address
3	Wrote the tiny app	The kitchen needs a recipe before it can cook
4	Told Ansible to build everything, and ran it twice	Twice is the proof it is safe to repeat
5	Took a photo (snapshot), broke things, rewind	Learn the undo button before you need it
6	Made a backup and restored it from scratch	Learn the ambulance before the accident
7	Put instruments on and ran a load test	You cannot tune an engine you cannot measure
8	Locked the doors	Do this last: a firewall mistake can lock you out
9	Wrote it all down	If it is not written down, it did not happen

Rule of thumb worth keeping: learn the undo and the safety net before you start breaking things on purpose. Module 5 and 6 before 8 is not an accident.

The numbers we ended up with

Short version, because Chapter 5 explains each one:

- Running the setup tool twice in a row: the second time it changed **nothing** (`changed=0`). That is the proof it is safe to run again and again.
- The site answers through the front door, and the kitchen door is **closed** to outsiders.
- The backup was deleted, then restored from the copy kept elsewhere, and every fingerprint matched: **RESTORE VERIFIED: 5 files match**.
- Deliberately corrupting a backup made the tool **refuse** it and exit with an error. A checker that never complains is not a checker.
- Adding processors: **1 → 2 → 4 processors** gave **17.9 → 29.2 → 44.1 requests per second**. Four times the cooks, 2.46 times the soup - and Chapter 5 explains why that is a good result, not a disappointing one.
- The restaurant was completely destroyed (the whole computer deleted) and rebuilt, serving again in **5 minutes 48 seconds**.

How to read the rest of this guide

Every subject below is explained the same four ways, so you can skip around:

- **What it is** - the plain idea, usually with an analogy.
- **Why we needed it** - the actual problem it solved in this project.
- **How we used it** - the concrete thing that happened here.
- **Where you will meet it again** - the real jobs and systems that use it.

Chapter 7 is a glossary and a command cheat sheet. If a word ever loses you, jump there.

2. Machines inside machines, and the wires between them

This chapter is about the ground floor: how you made two computers without buying two computers, and how they talk to each other.

A computer inside a computer (virtualization)

What it is. A program that pretends to be a whole computer. You give it some of your memory, some of your disk and some of your processors, and it behaves like a separate machine with its own operating system. You can start it, stop it, break it, and throw it away without touching the real computer.

Analogy. A folded paper house. It is a real house - you can live in it, put furniture in it - but it is standing inside a bigger room. Knock it down and the big room is fine.

Why we needed it. The project is about running servers. Real servers get installed, changed, backed up and destroyed. Doing that on your own laptop directly would be dangerous and messy. Virtual machines make it safe to experiment.

How we used it. We made two virtual machines: `web01` (the waiter) and `app01` (the kitchen).

Where you will meet it again. Almost everywhere. Any “cloud server” you rent is a virtual machine. Any company server room runs hundreds of them on a handful of big physical machines, because most servers sit idle and it is wasteful to give each one its own hardware.

The word for the boss program. The thing that creates and manages virtual machines is called a **hypervisor**. Three famous ones: **KVM** (what we used, free, part of Linux), **VMware vSphere** (what big companies often pay for), and **Hyper-V** (Microsoft’s). They all do the same job with different buttons.

KVM, qemu, and libvirt - three names for one job

Beginners get confused here because three programs are involved. Here is the division of labour:

Name	Plain meaning	In our project
KVM	The part built into Linux that lets a virtual machine use the real processor at full speed	It is a feature of the Linux kernel. We checked it was switched on.
qemu	The program that pretends to be the hardware: the disks, the network card, the screen	One <code>qemu</code> process runs for each virtual machine
libvirt	The manager. It keeps a list of machines, remembers their settings, and gives you one tidy set of commands	The <code>virsh</code> command you typed talks to libvirt
virsh	The command you type to talk to libvirt	<code>virsh list --all</code> shows your machines
virt-install	A helper that builds a new virtual machine in one command	<code>scripts/create-vms.sh</code> uses it

Analogy. KVM is the engine, qemu is the car body and wheels, libvirt is the garage manager who knows where all the cars are parked and what keys they need, and `virsh` is the clipboard you talk to the manager with.

Why not just use qemu directly? You can, but you would have to write a 40-line command every time and remember dozens of options. libvirt remembers for you, and gives you useful extras like snapshots and live resizing.

Where you will meet it again. `virsh` skills transfer directly. If you learn `virsh list`, `virsh start`, `virsh dominfo` and `virsh snapshot-list`, you will recognise the same shapes in VMware (`vim-cmd`, PowerCLI) and Hyper-V (`Get-VM`).

Why we ran Linux inside Windows (WSL2)

What it is. WSL2 is Microsoft's way of running a real Linux system inside Windows, sharing the same computer. It is not a pretend Linux; it is actual Linux in a lightweight virtual machine that Windows starts for you.

Analogy. A guest flat built into your house. Separate front door, separate kitchen, but the same building and the same plumbing.

Why we needed it. Your laptop runs Windows. KVM exists only in Linux. So we needed a Linux machine, and instead of installing one or buying a second laptop, WSL2 gave us one in about two minutes.

The important trick: nested virtualization. This means running a virtual machine inside a virtual machine - our KVM machines live inside the WSL2 machine. Windows has to agree to pass the processor's virtualization feature down the chain. It did, and we proved it:

```
sudo kvm-ok
# INFO: /dev/kvm exists
# KVM acceleration can be used
```

That `/dev/kvm` file is the doorway to the processor's virtualization feature. If it is missing, virtual machines still run but in **software emulation** - the computer pretends to be a different processor in slow motion. Everything works, just 10 to 50 times slower.

Honest cost. We are running inside a wrapper, so speeds are lower and the guest machines are capped by whatever WSL is given. The project's absolute speed numbers describe a laptop, not a data centre. Chapter 5 says more about why that still makes the experiment useful.

Where you will meet it again. WSL2 is extremely common now for developers on Windows. On servers you would use real Linux, but every command you learned here is the same command.

A ready-made computer: the cloud image

What it is. A file containing a complete, already-installed Linux system. Not an installer - a finished disk. You copy it and boot it, and 20 seconds later you have a working computer with no setup questions.

Analogy. A frozen meal versus cooking from scratch. Making Linux from an installer takes 20 minutes of clicking. The frozen version is ready in seconds, and you add your own salt afterwards.

Why we needed it. We needed to build the same machine twice - once normally, and once again during the disaster recovery test. Doing that with an installer would be slow and error-prone. With a cloud image it is one command.

How we used it. We downloaded Ubuntu's cloud image (597 MB) and **checked its fingerprint** against the publisher's published list before using it. That fingerprint check matters more than it looks: a corrupted or tampered base image is a problem you would not notice for months.

Where you will meet it again. Every cloud provider starts from images like this. "Amazon Machine Image" and "Azure Marketplace image" are the same idea.

cloud-init: the note you leave for a new computer

What it is. A way to hand a brand-new machine a small note on first boot saying: your name is this, make this user, here are the keys, turn off password logins.

Analogy. A note taped to the fridge in a new flat: "Your name is app01. The spare key belongs to Ziaur. Do not let anyone in without a key."

Why we needed it. A fresh cloud image has no user, no password and no way in. Without cloud-init you would have to open a console and type everything by hand for each machine.

How we used it. Our note created a user called `ops`, gave it permission to run administrator commands without a password (so Ansible can do its job), and installed our SSH key.

Where you will meet it again. Every cloud. It is the standard way to give a new server its identity. On the job you will write these notes, called `user-data` files, all the time.

Thick disks that are secretly thin (qcow2 and backing files)

What it is. A disk file format that only takes up space for the data you actually write. A “10 GB disk” may only use 700 MB on your real disk. It can also be stacked: a small disk on top of a big read-only one.

Analogy. Tracing paper. Instead of copying a whole drawing, you lay a sheet of tracing paper over it and only draw your changes. The original never gets touched, and your sheet is nearly empty.

Why we needed it. Two virtual machines each wanting a 10 GB disk would mean 20 GB of real space, most of it empty. With stacking, the 597 MB base is stored **once**, and each machine has a thin layer of its own.

How we used it. `create-vms.sh` does exactly this:

```
qemu-img create -f qcow2 -F qcow2 -b noble-server-cloudimg-amd64.img app01.qcow2 10G  
  
#                ^^ the base image                ^^ the thin sheet on top
```

Where you will meet it again. VMware calls it a **linked clone**, Hyper-V calls it a **differencing disk**, and Docker calls it a **layer**. Same idea, three brand names. This is one of the most transferable concepts in the project.

The ceiling you set when you build the machine

What it is. When you create a virtual machine you say two numbers: how much it has now, and the maximum it is ever allowed to have.

Why we needed it. Because “add a processor to a running machine” only works up to a limit fixed at creation. We wanted the machine to start small (so the load test would find a real bottleneck) and still be able to grow later.

How we used it. `app01` was created with **1 processor now, maximum 4**, and **2 GB of memory now, maximum 4 GB**. When we later tried to be greedy, the computer said no, politely and clearly:

```
$ virsh setvcpus app01 5 --live
error: requested vcpus is greater than max allowable vcpus for the live domain: 5 > 4

$ virsh setmem app01 8G --live
error: cannot set memory higher than max memory
```

Where you will meet it again. VMware and Hyper-V have the same concept - memory reservations and limits set when the machine is built. Interviewers like this question because it shows whether you have actually done it or only read about it.

The wires: a private network just for the machines

What it is. libvirt creates a small private network for its machines: `192.168.122.0/24`. Several things live in that one line.

Analogy. A private road behind the houses. The delivery van (the internet) can reach the road through the gate, and the houses can talk to each other easily.

Piece	Plain meaning	In our project
NAT network	Machines can reach the internet, the internet cannot reach in	Lets the machines install software
Bridge (<code>virbr0</code>)	The virtual switchboard the machines plug into	The thing that broke every time WSL restarted
DHCP	The automatic address giver	Hands out addresses when machines boot
DHCP reservation	"This machine always gets this address"	How <code>app01</code> is always <code>.11</code>
Gateway	The way out of the private road	<code>192.168.122.1</code> , the host itself

Why fixed addresses matter. Ansible's address book contains two addresses. If a machine woke up with a different one, every instruction would be sent to an empty house. The reservation is made against the machine's **MAC address** - the hardware's permanent serial number for its network card - so it does not matter what order machines start in.

Where you will meet it again. Every network you ever touch. Subnets, NAT, DHCP, gateways, MAC addresses - these concepts are in routers, cloud networks and office networks alike.

Getting in without a password (SSH keys)

What it is. A way to prove who you are using two matched files instead of a typed password: a **private key** you keep secret, and a **public key** you give away. The machine keeps a copy of your public key; if you can prove you hold the matching private key, you are let in.

Analogy. A padlock you hand out and the only key that opens it. You can give the padlock to a thousand people and they still cannot open it.

Why we needed it. Ansible logs in dozens of times per run. Typing a password each time is impossible, and passwords can be guessed. Keys cannot reasonably be guessed, and they let automation work.

How we used it. One key pair for the lab. The public half was installed by cloud-init; password logins were switched off entirely. We then proved it:

```
$ ssh -o PubkeyAuthentication=no ops@192.168.122.11
ops@192.168.122.11: Permission denied (publickey).
```

Where you will meet it again. Everywhere in servers and code hosting. The same idea is behind GitHub's push access, and behind every automated deployment you ever write.

The rule you must never break: the private key file never goes into the project folder and never into git. Our project automatically ignores files named like keys, and we checked that no key was ever committed.

3. The kitchen, the waiter, and the bouncer

Chapter 2 built two computers. This chapter puts food in one and a doorman on the other.

Java: the language the kitchen speaks

What it is. A programming language with one special trick: the same program can run on Windows, Linux or a Mac without being rewritten.

Analogy. Esperanto for computers. You write the recipe once, and every kitchen in the world can read it.

Why it matters. Java code does not run directly on the processor. It runs inside a program called the **JVM** (Java Virtual Machine), which translates as it goes. That middle layer is why Java is slower than some languages and why it works everywhere.

Where you will meet it again. Enormous amounts of business software are Java: banking, insurance, government systems. If a job mentions Tomcat, WebLogic, WebSphere, Spring or JBoss, the language underneath is Java.

Tomcat: the kitchen itself

What it is. A **web application server** for Java. It knows how to receive web requests, find the right piece of Java code, run it, and send the answer back.

Analogy. A kitchen with a rule: “Put your recipe box on this shelf and I will cook from it.” You do not hire cooks; you hand Tomcat a box and Tomcat cooks.

Why we needed it. Something has to actually run our code in response to a browser asking. Tomcat does that, plus a lot of unglamorous but essential work: managing many visitors at once, restarting a broken app, writing logs.

How we used it. We installed the `tomcat10` package, and Tomcat watches one folder: `/var/lib/tomcat10/webapps/`. Anything placed there gets picked up automatically.

Where you will meet it again. Tomcat is the most common Java application server in the world, and it is free. Its big commercial cousins are WebLogic (Oracle) and WebSphere (IBM) - same job, more buttons, much bigger price. If you can explain Tomcat, you can explain those in an interview.

JSP: a web page with code inside it

What it is. A file that looks like a page of text but can contain small bits of Java. Tomcat turns it into real program code the first time somebody asks for it.

Analogy. A recipe card with a note in the margin: “write today’s date here”. The card is mostly fixed text; the margin changes each time.

How we used it. Two tiny files:

```
health.jsp    -> prints: OK app01
work.jsp      -> spends about 87 milliseconds doing heavy arithmetic, then says
"done"
```

Why the slow one? Because of Chapter 5. To learn where a system gets stuck you need something that genuinely works the processor hard. A page that just prints text takes 1 millisecond and teaches you nothing. `work.jsp` was written to be deliberately expensive, and we tuned it until one request cost about 87 milliseconds - in the range a real, slightly heavy web page might cost.

A detail worth knowing. The very first request to a JSP is slow, because Tomcat has to compile it. That is why our Ansible task waits and retries instead of expecting success immediately. If you have ever wondered why a site is slow the first time and fast afterwards, this is often the reason.

Where you will meet it again. JSP is old-fashioned now - modern Java web apps use Spring Boot with templates - but the idea (a page with placeholders filled in by code) is everywhere. It is the same idea as PHP, or as templates in Python and JavaScript.

WAR: the recipe box

What it is. A single file containing a whole web application: pages, code, settings, all zipped up. The name means “Web Application aRchive”.

Analogy. A lunchbox. Everything the kitchen needs, in one box, with the label on the outside.

Why we needed it. It gives us one object to build, one object to copy, and one object to back up. That simplicity is the reason the whole deployment is a single step in Chapter 4.

How we used it. `app/build.sh` zips three files into `labapp.war`, and the file name decides the web address: `labapp.war` becomes the address `/labapp`. Tomcat unpacks it into a folder of the same name.

The little bug we found. Building the same source twice produced two different files. That sounds harmless but it broke our proof that the setup tool changes nothing on a second run - because the tool kept seeing “a different file!” and copying it again. The cause was extra hidden metadata that `zip` writes into archives. One extra flag (`zip -X`) fixed it. Chapter 6 tells the full story, because the lesson is bigger than the bug.

Where you will meet it again. WAR files are still how Java apps are shipped to Tomcat, WebLogic and WebSphere. Java’s newer sibling is the JAR (a plainer box), and other languages have their own versions: a `.whl` for Python, an `.npm` package for JavaScript. Same idea: one file, everything inside.

Apache: the waiter at the door

What it is. A **web server**: the program that receives web requests first. It is very good at the simple, high-volume work of greeting, checking, logging and handing requests onward.

Analogy. A restaurant's front-of-house staff. They do not cook, but they greet you, take your order, keep the guest book, and know that the kitchen is through the swing doors.

Why we needed it - and not just Tomcat? A fair question, and interviewers ask it. Four real reasons:

1. **A single entrance.** Visitors meet one machine, not several. From outside, the system looks simple.
2. **Security in one place.** When you eventually add HTTPS, you configure it once here. The kitchen never handles certificates.
3. **One guest book.** All access records in one file, which makes investigating problems much easier.
4. **Growing sideways.** If you add a second kitchen later, the waiter can share orders between them. Tomcat alone cannot do that.

reverse proxy: carrying the order to the kitchen

What it is. A rule that says "requests arriving here should quietly be handed to that other machine over there, and the answers brought back". The visitor never knows it happened.

Analogy. A phone answering service for a small business. You dial one number; a friendly voice answers and passes your message to whoever actually does the work.

How we used it. Two lines inside a virtual host file - the shortest and most important lines in the whole project:

```
ProxyPass          / http://192.168.122.11:8080/  
ProxyPassReverse   / http://192.168.122.11:8080/
```

Read it as: "anything asking for / should go to 192.168.122.11 on port 8080". The second line fixes up the answers on the way back so links work properly.

A virtual host is just a section of Apache's settings for one website. We disabled Apache's default one so only ours is active.

Where you will meet it again. Every serious website does this. The same job is done by nginx, HAProxy, and cloud load balancers. The term for the general idea is a **load balancer** when there are several kitchens, and a **reverse proxy** when there is one.

The bouncer: a firewall decides who may knock

What it is. A list of rules about which doors are open, to whom. Anything not allowed is ignored completely - the packets are dropped, so a visitor gets silence rather than a refusal.

Analogy. A bouncer with a guest list. Not on the list? You do not get told why; you simply do not get in. The door behind the building that everyone forgot about gets locked too - that is the part that matters.

Why we needed it. Before this step, the whole world (well, our little network) could knock directly on the kitchen door and skip the waiter. That made the waiter **decorative**. The project claimed a two-tier design, but nothing enforced it.

How we used it. We installed `ufw` (“uncomplicated firewall”) and wrote rules that match the design exactly:

On <code>app01</code> (the kitchen)	Who may knock
port 22	anyone (so we can administer it)
port 9100	only the monitoring machine
port 8080	only <code>web01</code> - the waiter

On <code>web01</code> (the waiter)	Who may knock
port 22	anyone
port 9100	only the monitoring machine
port 80	anyone - the only public entrance

Proof it worked. Before: knocking straight on the kitchen door returned “Hello, here is your soup” (HTTP 200). After: nothing at all.

```
through Apache:      HTTP 200  body=OK app01
app01:8080 from host: HTTP 000 (curl exit 28)
```

Exit code 28 means “timed out” - the packets were dropped, not refused. And we checked the other side too: the kitchen logged 10 blocked-connection entries. **A door that silently does not open is impossible to diagnose; a door that logs why is a door you can fix.** That detail is worth more in an interview than the rules themselves.

The dangerous part. A firewall mistake can lock you out of the machine you are working on. Our rules are written so all the “may come in” lines happen before the “everyone else is blocked” line. Reverse that order and the first thing the firewall blocks is the connection running the job that is configuring it. We also made sure a snapshot existed, and knew the console command to get back in, before running it.

Where you will meet it again. Constantly. Cloud providers call the same thing a **security group** or a **network ACL**, and they expect you to know which port needs

opening for which service. “Why can’t the servers talk to each other” is one of the most common problems in operations, and this is usually the answer.

4. Doing it automatically, and protecting it

Chapters 2 and 3 built the restaurant by hand. This chapter is about the manager with the checklist, the alarm clock, the photograph you can rewind to, and the copy of the recipe kept at grandma's house.

Why not just type commands like normal?

You could. Here is what happens when you do, three months later, on a second machine:

- You forget one of the 20 steps.
- You type it slightly differently, and something behaves slightly differently.
- You cannot tell whether the machine is correct or whether you just got lucky.
- You cannot show anyone what you did.

So instead of typing commands, we describe the **result we want** in a file, and let a tool make it true. The tool is Ansible.

Ansible: the manager with a checklist

What it is. A program that logs into machines and makes them match a written list. You write what you want - "Tomcat installed, this file here, this service running" - and Ansible works out the commands.

Analogy. A housekeeper with a checklist. You do not say "walk to the cupboard, pick up the cloth, move your arm in circles". You say "the kitchen should be clean". How to achieve it is their problem.

What is in the checklist, and the vocabulary:

Piece	Plain meaning	Our example
Playbook	The whole checklist	<code>site.yml</code>
Inventory	The address book of machines	<code>inventory.ini</code>
Role	A group of jobs for one purpose	<code>tomcat</code> , <code>apache</code> , <code>backup</code>
Task	One line on the checklist	"Install Tomcat 10"
Handler	A job that only runs if something actually changed	"restart Apache, but only if you edited its settings"
become	Run as the administrator	Needed to install software

Why we needed it. Two machines, dozens of settings, and a house rule that we must be able to rebuild everything after a disaster. Typing is not an option.

Why "the file decides the address" is a good thing. Read our inventory:

```
[app]
app01 ansible_host=192.168.122.11
```

Every instruction for the kitchen is written against the name `app01`. That is why the fixed address from Chapter 2 matters: names point at addresses, and if the address drifted, the instructions would go nowhere.

Where you will meet it again. Ansible is one of the most requested skills in operations job adverts. Its rivals are Puppet, Chef and Salt - same idea, and the concepts move across easily. Small setups often use plain scripts instead, but the moment there are more than a couple of machines, people reach for a tool like this.

Idempotent: the most important idea in the project

What it is. A long word for a simple property: **doing it twice is the same as doing it once.**

Analogy. A light switch, not a doorbell. Press a light switch twice and the light is on. Press a doorbell twice and you have rung it twice. Configuration should behave like the light switch.

Why it matters so much.

- You can run the checklist again without fear. It is always safe.
- You can run it after an interruption without unpicking what half-happened.
- You can use it to check a machine: run it, and if nothing changes, the machine is correct. That is a health check for free.
- It is the difference between “a script I ran once” and “a system I can manage”.

How we proved it. We ran the playbook twice and looked at the summary:

```
PLAY RECAP
app01  : ok=18   changed=0    failed=0      <- ran 18 jobs, changed nothing
web01  : ok=17   changed=0    failed=0
```

Read that carefully, because there is a subtlety worth understanding. `ok=18` with `changed=0` does **not** mean the jobs were skipped. It means all 18 jobs ran, looked at the machine, and reported “this is already how you asked it to be”. `failed=0` means none of them broke.

If a job is skipped you see `skipped`, which is a different thing entirely. If a job does something you see `changed=1`. **`changed=0` is the proof.**

A trap we fell into. Our proof was quietly false for a while, because one job kept reporting `changed`: we were rebuilding the app file, and the rebuild produced a slightly different file every time. So Ansible honestly said “this file is different, I copied it again”. The checklist was fine; the thing we handed it was not stable. Chapter 6 has the full story.

Where you will meet it again. In every interview about automation, and in every real system. The word, the idea and the proof are all expected knowledge.

Git and GitHub: a notebook that remembers everything

What it is. Git keeps the entire history of a folder - every change, who made it, and why. GitHub is a website that holds a copy of that history so others can see it, and so you can work on it from anywhere.

Analogy. A lab notebook where you cannot erase. You can write a new page saying “actually, yesterday’s page was wrong”, but yesterday’s page stays there for anyone to read. That is a feature, not a bug.

Why we needed it. Three copies of safety: the project can be rebuilt from it (which is exactly what the disaster recovery test proved), the history explains why things are the way they are, and it is the thing a future employer looks at.

The workflow we used, and why it looks fussy. Every piece of work went on its own **branch** and came back through a **pull request**:

1. Make a branch - a private copy of the work, like a rough draft.
2. Do the work, in several small saved steps (**commits**).
3. Ask for it to be added to the main version (**pull request**, or PR).
4. Look at the change one last time, then merge it in.

Analogy. Nobody redraws the class poster by rubbing bits out while everyone watches. You draw your idea on a separate sheet, show it, and only then stick it on the poster.

Even working alone, this helps: the PR is a pause where you look at your own work as a stranger would. It is also exactly how every software team works, so practising the shape now means it is familiar later.

A small habit worth copying. Every commit message says what changed and why, not just “update”. Three months from now, “why” is the only part you will need.

systemd timers: the alarm clock

What it is. Linux’s built-in scheduler. You describe a job and when it should happen, and Linux runs it for you.

Analogy. An alarm clock next to a note that says “when the alarm goes off, make a backup”.

Why we needed it. A backup you have to remember is not a backup. Ours runs itself every night at 2am.

Two units, not one. The note (`.service`) says what to do; the alarm (`.timer`) says when. That separation is a Linux convention.

A detail worth understanding. We set `Persistent=true`, which means: if the machine was switched off when the alarm should have rung, run the job as soon as it wakes up. Our

lab machines are frequently off, so without this a night's backup would silently not happen. **Silently not happening is the worst behaviour a backup can have.**

Where you will meet it again. Timers and their older cousin **cron** are on every Linux server. Anything that must happen regularly - backups, cleanups, reports, certificate renewals - runs this way.

tar and gzip: the box

What it is. `tar` packs many files into one file. `gzip` squeezes that file to make it smaller. Together they make `.tar.gz`, one tidy box.

Analogy. Putting the whole kitchen's recipe cards into one box, then vacuum-sealing the box.

Why we needed it. One object is far easier to copy, verify, move and store than a folder of loose files.

Where you will meet it again. Constantly. Nearly every backup, download and software package you handle is a `.tar.gz` underneath. (`.zip` is the Windows cousin and does the same job.)

SHA-256: the fingerprint

What it is. A recipe that turns any file into a short string of letters and numbers - its fingerprint. Change one letter in the file and the fingerprint changes completely. You cannot work backwards from the fingerprint to the file.

Analogy. A fingerprint. It says that this is the same person, without telling you anything else about them. Cut your hair and the fingerprint still matches; change your identity and it will not.

Why we needed it. Fingerprints are how we answer the question "is this exactly the same as that?", which is the heart of every check in this project. Two places in particular:

- **Is the backup really the same as what we backed up?** Compare fingerprints.
- **Did this file survive the trip to another machine?** Compare fingerprints.

Where you will meet it again. Everywhere in modern computing. Downloads, code hosting, package managers, blockchains, git itself. The name is worth understanding: SHA-256 is one specific fingerprint recipe, and it is the common one.

A manifest: the list of fingerprints

What it is. A file listing the fingerprint of every file inside a backup.

Analogy. An inventory list taped to the box, saying "inside: 5 items, and here is each one's fingerprint".

Why we needed it. Without it, a backup is a box you have never opened. With it, you can prove the contents later, on a different machine, without having the original to compare against.

How we used it. Our backup tool writes this list as it packs the box. Later questions like “does this restore contain the right things?” become a simple comparison of two lists.

Where you will meet it again. Software vendors publish checksum lists so you can check a download. Package managers keep manifests of installed files. It is the same trick.

The backup that checks itself

This is the part of the project worth the most in an interview, and it is only about 30 lines of shell script. Here is the pattern:

```
pack the folder into a box
-> open the box into a temporary scratch folder
-> fingerprint everything found inside
-> compare against the list of fingerprints taken from the original
-> different? shout "RESTORE MISMATCH" and exit with an error
```

Why not just fingerprint the box itself? Because that only proves the box did not rot while sitting on the shelf. It proves nothing about whether the box contains your application. Opening it and comparing the contents answers the real question: if I unpacked this on a brand-new machine, would I get my app back?

Why the failure drill matters. A checker that only ever prints “fine” is useless. So we deliberately corrupted a backup and demanded the tool refuse it. It did:

```
RESTORE MISMATCH: /var/backups/tomcat/webapps-...tar.gz
exit code: 1
```

Then we put it back and demanded it pass again - so we know the tool can tell the difference, rather than always complaining.

Where you will meet it again. “We have backups” is a claim. “We have restored a backup this quarter” is evidence. Employers have been burned by the first kind of claim, so the second kind gets you hired.

rsync: the careful mover

What it is. A tool for copying files between machines that only transfers what has changed, and can pick up where it left off.

Analogy. A removal van that checks which boxes are already at the new house and only carries the rest.

Why we needed it. A backup that lives only on the machine it protects is not a backup - it dies with the machine. We copy the boxes to the host machine and **check the fingerprints again after the journey**, because a copy that silently went wrong is worse than no copy, since you think you are safe.

The honest limitation. Our copy sits on the same physical laptop. It survives the virtual machine dying - which the disaster recovery test proved - but not the laptop dying. Real systems send copies to a different building or to cloud storage. Saying that out loud is part of being trustworthy about a system.

Where you will meet it again. rsync is one of the most-used tools in Linux operations. Its cousins are `scp` (plain copy over SSH) and cloud storage tools.

Snapshot versus backup: the photograph and the copy

This is the question interviewers ask most often, so learn the difference properly.

	Snapshot	Backup
Analogy	A photograph of the room	A copy of the recipe kept at grandma's house
Where it lives	On the same disk as the machine	Somewhere else
Good for	Undoing a mistake in seconds	Surviving a disaster
Survives losing the disk?	No - it dies with it	Yes, that is the whole point
Contains memory?	Optionally yes, as ours did	No, it is a file archive
Is it checked?	Not applicable	Restored and fingerprinted

How we proved the difference instead of just claiming it. In the disaster recovery test we deleted the machine with its disk. The snapshot went too. Nothing about the snapshot could have saved us; only the backup could.

The snapshot drill we ran. Take a photograph, then deliberately wreck the kitchen, then rewind:

- Deleting the app: the site went to **404** ("the kitchen is fine, but the recipe is missing").
- Rewinding: **5.1 seconds**, and the app came back with the identical fingerprint.
- The clock test: the machine's uptime read "up 1 minute" before the rewind and "up 1 minute" after it, proving the rewind did **not** reboot the machine - it restored a running moment. That is why it takes five seconds instead of the three minutes a boot costs.

Reading error numbers like a detective. 404 and 503 look similar and mean different things:

- **404** = the kitchen is alive and well, but the recipe is missing.
- **503** = the kitchen itself is not answering.

Knowing which one you are looking at saves an hour of hunting in the wrong place. This exact distinction cost us time during the build, in Chapter 6.

Growing a running machine (live resize)

What it is. Adding memory or processors to a machine that is switched **on**. No reboot, no downtime.

Analogy. Laying another table for a dinner party while the guests are eating, rather than asking everyone to leave and come back.

Why we needed it. Because it is how real capacity planning works: you measure, then you add, then you measure again. And because the maximum you are allowed to grow to must be set when the machine is created (Chapter 2).

Memory worked instantly. The machine saw the new memory immediately, with no reboot at all: 1867 MB became 2891 MB inside the guest.

The processor arrived but stayed asleep. This is the best small discovery in the project. libvirt said “you now have 2 processors”. The machine still said “I have 1”. The machine’s own log said:

```
ACPI: CPU1 has been hot-added
```

So the processor was delivered and plugged in - and then left switched off. The number you check (`nproc`) counts processors that are awake. One extra command to switch it on, and `nproc` said 2.

Why this is worth telling an interviewer. It shows the difference between “the tool said it worked” and “I verified the thing actually works”. Many people stop at the first one.

Where you will meet it again. VMware calls it hot-add, Hyper-V has dynamic memory. The same surprises happen, because they come from the same place: the guest operating system has to notice and accept the change.

5. Watching, measuring, and finding the slow part

Chapter 4 protected the restaurant. This chapter puts instruments on it and works out where the queue forms.

Why measure at all?

What it is. Writing down numbers about a system over time, so you can answer questions instead of arguing about opinions.

Analogy. A thermometer and a chart on the kitchen wall. Without them, “the kitchen feels slow today” is the best anyone can say. With them, you can see that the kitchen hits its limit every day at noon, and does nothing about it for the other 23 hours.

Why we needed it. Three jobs:

1. **To see problems before people complain.** “Is the machine about to fill its disk?” is answerable in advance.
2. **To find the slow part.** Guessing where a system is stuck is usually wrong.
3. **To prove a change helped.** We added processors and could show exactly what it bought us, in numbers.

Where you will meet it again. Every operations job. The general term is **observability**, and it splits into three related ideas worth knowing by name: **metrics** (numbers over time), **logs** (things that happened), and **traces** (the path one request took through a system).

Prometheus: the nurse with the notebook

What it is. A program that asks every machine “how are you?” every few seconds, writes down the answers with a timestamp, and lets you ask questions about the history.

Analogy. A nurse who walks the ward every 15 seconds and writes down everyone’s temperature. Nobody has to be watched; the notebook fills itself.

Why we needed it. Somebody has to collect the numbers, and importantly, keep them. A number you looked at once is forgotten. A number written down every 15 seconds lets you see that the processor has been busy since 9am.

Two words worth knowing. Each machine “answers questions” at a web address - ours at port 9100. Prometheus **scrapes** that address, which simply means it fetches the numbers on a timer.

Where you will meet it again. Prometheus is the standard free monitoring system. Its hosted cousins are Datadog, New Relic and Grafana Cloud. The words “scrape”, “metric” and “alert rule” transfer directly.

node_exporter: the little meter on each machine

What it is. A small program installed on a machine that reports basic facts about it: how busy the processor is, how much memory is free, how full the disk is, how much network traffic there has been.

Analogy. A meter reader for a house. It does not fix anything; it just knows how much electricity is being used and tells anyone who asks.

Why we needed it. Prometheus cannot see inside a machine by itself. It needs something inside to report. That is all `node_exporter` is.

A useful detail. It is not part of the operating system; it is just a program. If it stops, monitoring of that machine stops - so one of our alert rules is literally “this machine has stopped answering”.

Where you will meet it again. Almost every Linux server being monitored has one of these installed. The pattern (a small agent reporting local facts) repeats everywhere: container agents, database exporters, application exporters.

A metric: a number with a timestamp

What it is. The basic unit of monitoring: a number, a name, and the time it was taken.

Analogy. A row in the nurse’s notebook: “3:15pm - Mr Patel - 37.1”.

Why it matters that numbers come with timestamps. Because a single number is almost meaningless and a series of them is a story. “Processor at 100%” is scary. “Processor at 100% for two minutes and then back to 5%” is a build job. The history is the information.

PromQL is the little language for asking questions of these numbers. Our whole alerting setup uses expressions a person can read aloud, for example:

```
100 - (average idle processor percentage) > 80
```

That reads as: “if the machine has been more than 80% busy for two minutes, tell me”. The “for two minutes” part matters - a brief spike is normal, so rules wait before shouting. Learning to write that short delay is the difference between alerting that people trust and alerting that people ignore.

Where you will meet it again. Every monitoring system has a query language. Learning one teaches you the shape of them all.

Alert rules: the rules that decide when to say something

What it is. Small statements that turn numbers into meanings: “if this, for this long, then that is a problem”.

Analogy. A smoke alarm. Not a fire - a rule about fires, so nobody has to sit and watch.

Why we needed them. Nobody watches a chart for a living. Rules watch the chart and only bother a human when something is genuinely wrong.

How we used them. Four rules, all readable at a glance:

Rule	Written as	Plain meaning
InstanceDown	<code>up == 0</code> for 1 minute	A machine has stopped answering
HighCPU	processor over 80% for 2 minutes	Something is working very hard
LowDisk	less than 15% of the disk free	The disk is filling up
LowMemory	less than 10% memory available	Memory is nearly used up

Note which ones are not about the current moment. A processor spike is not news. A processor that has been flat out for two minutes is. Good alerts describe a trend, not an instant.

Where you will meet it again. Every system that pages someone at 3am has rules like these. Writing alerts that do not cry wolf is a skill in itself, and “alert fatigue” - where people ignore alerts because there are too many - is a well-known problem.

Grafana: the chart on the wall

What it is. A program that draws Prometheus’s numbers as pictures, and lets you arrange those pictures on a page.

Analogy. The nurse has the notebook; Grafana draws the chart on the wall so everyone can see at a glance how the ward is doing.

Why we needed it. Numbers in a table are slow to read. A line going up is instant. Pictures are also how you explain a system to someone else in a meeting.

A dashboard is just a page full of charts. We used a well-known ready-made one called “Node Exporter Full” (number 1860 on grafana.com), which draws everything about a Linux machine - processors, memory, disk, network - without us designing it.

One thing we did differently from most tutorials. Rather than clicking in the web page to connect Grafana to Prometheus, we put the settings in a file in the project. So when someone downloads this project, the charts are already connected - nothing depends on a person remembering to click. **Making setup automatic instead of “here are the steps, don’t forget them” is worth more than it sounds, and interviewers notice it.**

Where you will meet it again. Grafana is extremely common for dashboards in every industry. Being able to build a useful dashboard - not a pretty one, a useful one - is a genuinely valuable skill.

Docker: shipping containers for software

What it is. A way to run a program together with everything it needs, sealed inside a standard box, so it works the same on any machine. Running several such boxes together is arranged with a file called `docker-compose.yml`.

Analogy. Shipping containers. It does not matter what is inside, the same crane can lift it and the same ship can carry it.

Why we needed it here. Prometheus and Grafana are two programs with their own requirements. Using containers meant two lines of setup instead of two software installations.

A networking detail that mattered. Containers normally live in their own little network and cannot see the virtual machines. We used a setting called **host networking** to put them on the same network as the machines, and then checked it worked before trusting it, by asking a container to fetch a page from one machine. Assumptions in networking are how afternoons disappear.

Where you will meet it again. Everywhere, and probably in your next job. The ideas to learn next after this project are how containers store data, and Kubernetes, which is the tool for running many containers across many machines.

ab: pretending to be a crowd

What it is. A small program that sends many web requests at once, to see how the system behaves under pressure. The name means ApacheBench.

Analogy. Hiring 20 people to call the restaurant at the same moment, to find out whether one waiter is enough.

Why we needed it. "Is this fast?" is not answerable by clicking a page once. You need many visitors at the same time, repeatedly, with numbers recorded.

The command, and what it means:

```
ab -n 2000 -c 20 http://192.168.122.12/labapp/work
```

- `-n 2000` means "send 2000 requests in total".
- `-c 20` means "20 of them at the same time".
- The address is the deliberately expensive page from Chapter 3, because we want the system to work hard, not idle.

Where you will meet it again. Load testing is standard practice before a big launch. `ab`'s more powerful relatives are `wrk`, `hey`, `JMeter` and `k6`.

The two numbers that matter: throughput and p95

Throughput (also “requests per second”) is how much work the system finishes per second.

Analogy. How many bowls of soup the kitchen serves per minute.

p95 latency is the time 95 out of every 100 requests took. Not the average - the slow ones.

Analogy. The average waiting time might be one minute, but if one guest in twenty waits fifteen minutes, that guest is writing an angry review. The average hides them; p95 finds them.

Why p95 instead of the average. Averages let a few terrible results hide behind lots of quick ones. Real users experience the slow ones. This is why real teams watch p95 and p99, and saying so in an interview marks you as someone who has actually looked at a dashboard rather than just read about one.

What we found, and why the answer was interesting

We measured the system with 1, then 2, then 4 processors on the kitchen machine:

Kitchen processors	requests/sec	p95	kitchen CPU	waiter CPU	whole laptop CPU
1	17.93	1497 ms	100%	14%	16%
2	29.18	971 ms	100%	14%	31%
4	44.14	761 ms	100%	14%	56%

The disappointing-looking headline: four times the processors gave only 2.46 times the soup. Not four times.

Why that is actually the good result. Because of what the numbers rule out:

- The kitchen is pinned at **100%** every single time. So the kitchen really is the thing holding everything back. Good - we found it.
- The waiter stays at **14%**. So the front door is not the problem, and 44 soups per second is not an artefact of the waiter being slow.
- The whole laptop never goes above **56%**. So we did not “run out of computer”, and the crowd-generating program was not itself the limit.
- The laptop has **4 real processor cores**, each pretending to be two. At 4 processors, the kitchen is using every real core - and sharing them with the waiter, the nurse (Prometheus), the chart (Grafana) and the crowd itself. Sharing a core with someone else slows you down, and that is why the third and fourth processor add less than the first and second.

What we genuinely could not tell apart. Whether the flattening is caused by core sharing, or by the Java program’s own internal bottlenecks (some parts of a program

simply cannot be parallelised). Both produce exactly the same shape of curve. Saying “I cannot separate those two with the data I have” is a stronger answer than inventing a reason.

Being honest about the numbers. We ran the experiment twice and got 14.71 / 23.88 / 36.65 the first time and 17.93 / 29.18 / 44.14 the second. That is a consistent 20% difference across all three points - which means something systematically changed between runs, rather than random wobble. So the shape is the finding, and the exact numbers are approximate. **Quoting 44.14 as if it were precise would be dishonest, and honesty about measurement is a professional skill, not a weakness.**

Capacity planning: how many cooks do we need?

What it is. Using measurements to decide how big a system should be, instead of guessing.

Analogy. Deciding how many tables to put in a restaurant by counting how many guests actually arrive, rather than by what feels about right.

The idea in three steps, which is really the takeaway from this whole chapter:

1. **Measure** the system as it is now. (One cook: 17.9 soups per second.)
2. **Change one thing** and measure again. (Two cooks: 29.2. Four: 44.1.)
3. **Find the limit.** Where does adding more stop helping, and what runs out first?

Why step 3 is the valuable one. Adding cooks forever is expensive, and past a point it does nothing at all. Knowing where the ceiling is - and being able to say why - is what separates “I ran a load test” from “I did capacity planning”.

Where you will meet it again. Every company that pays for computers. Cloud bills are directly tied to how big you make things, and being the person who can say “we need this size, and here is the measurement” is genuinely valuable to an employer.

6. Everything that broke, and what it teaches

This is the most valuable chapter in the guide. A project where nothing broke teaches you almost nothing, and an interviewer knows it. Every story below is real, in the order it happened.

Each one has the same five parts: what we saw, what we guessed, what it really was, how we fixed it, and the lesson.

Story 1: The bridge that would not come back

What we saw. Every time Windows' WSL restarted, the private network for our machines refused to start:

```
error: Failed to start network default
error: internal error: Network is already in use by interface virbr0
```

What we guessed. A broken configuration. Maybe a wrong setting somewhere.

What it really was. Remember the bridge from Chapter 2 - the virtual switchboard called `virbr0` that the machines plug into. When WSL restarts, it does not restart the whole kernel; it restarts the Linux user space on top of the same kernel. The switchboard lives in the kernel's part, so it survived. The program that manages it (libvirt) did not. So the new manager woke up, saw a switchboard it had not created, and refused to touch it. It was being cautious, not broken.

How we fixed it. Delete the orphaned switchboard before the manager starts:

```
sudo ip link delete virbr0
virsh net-start default
```

Then, because it happened every single time, we made it automatic with a small script that runs just before the manager starts.

The lesson. When a message says "already in use", ask who is using it and why the other thing does not know about it. The word "already" is a clue that something survived when it should not have. Also: this problem does not exist on a normal Linux server, because a real reboot destroys everything together. **Knowing that your environment has quirks, and writing them down, is part of the job.**

Story 2: The guard that could never say "no"

What we saw. Nothing. That is what makes this story dangerous.

What happened. The automatic fix from Story 1 needs a safety check. It must delete the switchboard only when no machines are running - otherwise it would rip the network out from under a working machine. So we wrote:

```
if no qemu process is running; then delete the switchboard; fi
```

And we wrote the check as `pgrep -x qemu-system-x86_64`.

What it really was. Linux shortens every program's name to **15 characters**. The program is called `qemu-system-x86_64`, which is 18. So its name is stored as `qemu-system-x86`. Our check was looking for a name that never exists, so it **never found anything**, so the answer to "is a machine running?" was always "no". Which meant the guard was not a guard at all: it would have deleted the network switchboard even with machines running, pulling the rug from under them.

How we fixed it. Ask a question whose answer cannot be wrong: does the switchboard have anything plugged into it?

```
if the switchboard has no ports attached; then delete it; fi
```

That is the actual state of the world, not a guess based on a name.

The lesson. A check that can never fail is not a check. It looks right in review and it is catastrophic in production, because it gives you the feeling of safety without the substance. Also, this is why we tested it against a live machine instead of trusting it. The most dangerous line of code in any system is the one that decides whether to do something destructive.

Story 3: It changed nothing, and it was lying

What we saw. Our proudest result was that running the setup tool twice changed nothing the second time. Then we noticed the deploy job reported a change **every** time - even when we had changed nothing at all.

What we guessed. A bug in Ansible. Or a file permission problem.

What it really was. We rebuilt the app file, and the rebuild produced a different file even though the three source files were identical. The cause: `zip` writes extra hidden information (high-precision timestamps and similar) into archives. Different bytes meant Ansible honestly reported "this file is different, I copied the new one".

How we fixed it. One extra flag tells `zip` to leave that extra information out:

```
zip -q -X -r app.war .  
#      ^ this one
```

Two rebuilds now produce identical fingerprints, and the second run of the setup tool reports `changed=0` properly.

The lesson. An idempotency claim is only as good as the thing you feed it. We had tested the tool; we had not tested whether the input was stable. If a thing that should

be identical is not identical, every clever check built on top of it is meaningless. This is why the project now prints fingerprints during builds - it turns an invisible problem into a visible one.

Story 4: The backup of nothing, which verified perfectly

What we saw. Nothing yet. This one we caught by thinking, not by failing.

The problem. Our backup tool packs a folder, writes fingerprints for everything inside, then opens the box and checks the fingerprints match. If the folder is **empty**, then the list of fingerprints is empty, the restored list is also empty, and the two empty lists are identical. The tool would announce:

```
RESTORE VERIFIED: 0 files match
```

A backup of nothing, reporting perfect health, every night, forever.

Why this was not hypothetical. During the build, our application folder really was empty for a while. A tool without this guard would have “successfully” backed up a week of nothing, and we would have found out during an emergency.

How we fixed it. Four lines at the top of the tool: if the folder has no files, refuse to run and say why.

The lesson. A checker must be able to fail, and you must have thought about what “empty” means. Most systems do not fail loudly; they fail quietly and convincingly. Whenever you write a check, ask: what input would make this check pass while being completely wrong?

Story 5: The command that printed nothing

What we saw. We had deleted the application folder on purpose (as part of the snapshot drill), but when the tool listed the folder before deleting it, the list was **empty**. That made no sense - the app had been running seconds earlier.

What we guessed. Something had lost our files. We spent several turns on a theory about data being lost during a hard shutdown, because the machines had been restarting unexpectedly at the time.

What it really was. Boring. The connection to the machine dropped mid-command, and we had told the command to throw away its error messages:

```
ssh ... "ls /var/lib/tomcat10/webapps/" 2>/dev/null  
#                ^^^^^^^^ this
```

The error message went into the bin. All we saw was silence - and **silence looks exactly like an empty folder**.

How we fixed it. Stop hiding errors on the commands that are supposed to prove something. And the real fix was mental: after that, every destructive step in the project

was bracketed by fingerprint checks, so “the file is missing” could never be confused with “the command did not run”.

The lesson. Never silence the error output of the command you are using as evidence. The first theory had been wrong, and it cost hours. The fix was not clever: it was to make the evidence unambiguous. Two rules worth keeping forever: do not throw away error messages, and never let “no output” count as proof.

Story 6: The machines kept rebooting themselves

What we saw. The virtual machines rebooted every few minutes. Our work kept vanishing. Commands failed with “no route to host”. It looked like the lab was falling apart.

What we guessed, and ruled out one by one. This is worth reading as a method, because the guessing is what took the time:

- The idle timer. WSL switches itself off when nothing is happening. We turned that off and told it never to do it. **Not the cause.**
- The memory killer. Linux has a service that kills programs when memory runs out (`systemd-oomd`). Check whether it killed our virtual machines. **It was not even installed.**
- The watchdog. Each virtual machine has a “dead man’s handle” device that reboots it if it stops responding. If it were armed, it would reboot under load. We checked whether anything was arming it. **Nothing was.**
- The manager crashing. We looked at how many times libvirt had crashed. **Zero times.**

What it really was. We finally looked at the kernel’s own log and saw the same boot messages repeating every few minutes:

```
Linux version 6.18.40.1-microsoft-standard-WSL2 ...  
WSL (1 - mini_init): WSL user cgroup created ...
```

but at the same time, the machine’s **uptime** and its **boot identity** said “nothing has restarted”. Both facts were true, and together they were the answer: WSL keeps its kernel running, but **tears down the Linux user space as soon as the last session closes**. Closing a session kills the virtual machine manager, which takes the virtual machines down with it. Our machines only came back because we had told them to start automatically.

So the “reboots” were not crashes at all. They were the lab being switched off every time we stopped typing, and switched back on when we typed again.

How we fixed it. Keep a session open so WSL never decides to tear things down:

```
nohup wsl.exe -e bash -c "sleep 3600" >/dev/null 2>&1 &
```

Then we verified it: we watched the process start times and the manager’s start time, waited 90 seconds doing nothing, and looked again. Unchanged. Fixed.

The lesson. “The tool said it restarted” and “the machine restarted” are different claims. The breakthrough came from noticing two facts that could not both be

true, and treating that contradiction as the clue instead of ignoring it. And the method note matters too: we ruled out four plausible causes before finding the real one, which is normal. Debugging is mostly the discipline to eliminate without getting bored.

Story 7: The processor arrived, but stayed asleep

What we saw. We added processors to a running machine. The manager reported success: “2 processors”. The machine itself said “I have 1”.

What we guessed. The change had not really happened.

What it really was. The machine’s log held the answer:

```
ACPI: CPU1 has been hot-added
```

The processor had been delivered and plugged in - and left **switched off**. The command we were using to count processors counts the ones that are awake. One extra command to switch it on, and the count was correct.

Why nobody documented it. Most guides check “did the tool report success?” and stop. Success at the top does not guarantee success all the way down.

How we fixed it. Switch the processor on explicitly before measuring, and document that the obvious verification step is not enough.

The lesson. Verify at the place the work is used, not where it is issued. The manager’s opinion is not evidence. The machine’s own report is.

Story 8: The clock that went backwards

What we saw. After we rewound a machine to a snapshot, its clock jumped backwards. The machine noticed and said so in its log. About a minute later, it restarted on its own.

What we guessed. Something was badly wrong - possibly the watchdog from Story 6, which would have been genuinely serious, because a watchdog firing during a load test would silently ruin all our measurements.

What it really was. Rewinding to a snapshot restores a moment, including the clock reading from that moment. And it invalidates anything the machine was in the middle of doing at that moment, such as a write to disk. The machine notices both, and can decide to restart to be safe. Not a fault - a side effect of time travel.

How we fixed it. Nothing to fix, but we checked the scary possibility properly: it was not the watchdog. That mattered, because if it had been armed, it could have restarted a machine in the middle of measuring, and we would have published wrong numbers without knowing.

The lesson. When you use time travel, expect the clock and unfinished work to be wrong. And: distinguish “harmless side effect” from “the same symptom caused by something that would ruin my results”. We spent a few minutes on the second question and it was the right few minutes.

The five lessons worth memorising

If you remember nothing else from this chapter:

1. **A check that can never fail is not a check.** Test your safety net by trying to break it, with a live system.
2. **A claim is only as good as the thing you are claiming about.** If a thing that should be identical is not, everything built on it is meaningless.
3. **Never silence errors on a command that is supposed to prove something.** “No output” is not evidence of anything.
4. **The tool saying it worked is not the same as it working.** Verify where the work is used.
5. **Write it down.** Every story here is in the build log with the commands and the output. A problem you solved and did not record is a problem you will solve twice.

7. Glossary, cheat sheet, and what to learn next

Glossary, in plain words

Ansible - A program that reads a written checklist and makes machines match it. The manager with the clipboard.

Apache - The waiter. The web server that greets requests first and passes them to the kitchen.

ab - ApacheBench. A program that pretends to be many visitors at once, to see how a system behaves under pressure.

backing file - The big read-only disk that thin disks are layered on top of, like tracing paper over a drawing.

backup - An independent copy kept somewhere else, which is checked. The copy of the recipe at grandma's house.

bridge (virbr0) - The virtual switchboard that virtual machines plug into.

capacity planning - Deciding how big a system should be by measuring it, rather than guessing.

checksum / fingerprint (SHA-256) - A short string computed from a file. Change one letter and it changes completely. Used to answer "is this the same as that?"

cloud image - A finished, pre-installed operating system in a file. A frozen meal instead of cooking from scratch.

cloud-init - The note you leave for a brand-new machine telling it who it is and how to let you in.

commit - One saved step in the history of a project, with a message about why.

container (Docker) - A program sealed together with everything it needs, so it runs the same anywhere. A shipping container.

cron / timer - The alarm clock that runs jobs on a schedule.

DHCP - The automatic address giver on a network.

DHCP reservation - "This machine always gets this address", matched by the machine's hardware serial number.

emulation - Pretending to be a different processor, in slow motion. What happens when hardware virtualization is unavailable.

firewall (ufw) - The bouncer with the guest list. Decides which doors are open to whom.

Grafana - The chart on the wall. Draws monitoring numbers as pictures.

guest - A virtual machine, from the point of view of the host.

handler - An Ansible job that only runs if something actually changed. "Restart the service, but only if you edited its settings."

host - The real machine that runs virtual machines; or, the thing a container runs on.

hot-add / live resize - Adding memory or processors to a machine that is switched on.

hypervisor - The boss program that creates and manages virtual machines. KVM, VMware, Hyper-V.

idempotent - Doing it twice has the same result as doing it once. A light switch, not a doorbell.

inventory - Ansible's address book of machines.

JAR / WAR - The sealed box containing a Java program (`.jar`) or a whole Java web application (`.war`).

Java / JVM - The language and the program that runs it. The Esperanto of programming languages.

JSP - A web page with small pieces of Java code inside it.

KVM - The virtualization feature built into Linux. The engine.

libvirt - The manager that remembers virtual machines and gives you tidy commands for them. Talked to with `virsh`.

manifest - A list of the fingerprints of every file in a backup. The inventory list taped to the box.

metric - A number with a timestamp. A row in the nurse's notebook.

NAT - A private network that can reach out to the internet, while the internet cannot reach in.

node_exporter - The little meter on each machine that reports how busy it is.

p95 latency - The time 95 out of 100 requests took. Finds the slow ones that an average would hide.

playbook - The whole checklist in Ansible (`site.yml`).

Prometheus - The nurse with the notebook. Collects and stores monitoring numbers, and checks alert rules.

PromQL - The little language for asking questions of monitoring numbers.

pull request (PR) - A polite request to add your work to the main version, with a chance to look at it first.

qcow2 - The disk file format that only uses space for data you actually write, and can be layered on a base image.

qemu - The program that pretends to be the hardware of a virtual machine. The car body and wheels.

reverse proxy - A rule that quietly hands requests to another machine and brings the answers back. A phone answering service.

role - A group of Ansible jobs for one purpose, such as `tomcat`.

rsync - The careful mover that copies only what changed and can be interrupted.

scrape - Prometheus fetching numbers from a machine on a timer.

snapshot - A photograph of a machine at one moment, stored on the same disk. For fast undo, not for disasters.

SSH key - A padlock you hand out and the only key that opens it. Replaces passwords.

systemd - The part of Linux that starts and supervises programs.

task - One line on the Ansible checklist.

throughput - How much work is finished per second. Bowls of soup per minute.

Tomcat - The kitchen. A Java web application server.

virtual host - A section of a web server's settings for one website.

virtualization - Making a computer inside a computer. A folded paper house inside a bigger room.

virsh - The command you type to talk to libvirt. Talking to the garage manager.

WSL2 - Microsoft's way of running real Linux inside Windows. A guest flat built into your house.

Command cheat sheet

Machines

```
virsh list --all           # list machines and whether they are running

virsh dominfo app01        # full detail: processors, memory, state
virsh start app01 / shutdown app01 # switch on / ask politely to switch off
virsh destroy app01        # pull the plug (use with care)
virsh console app01        # look at its screen (leave with Ctrl + J)

virsh net-list --all       # is the private network up?
virsh net-dhcp-leases default # which machine has which address
virsh domblklist app01     # which disk file is it using
virsh autostart app01      # start it automatically when the host boots
```

Snapshots and resizing

```
virsh snapshot-create-as app01 my-photo --description "..."
virsh snapshot-list app01
virsh snapshot-revert app01 my-photo
virsh setvcpus app01 2 --live # add processors while running
virsh setmem app01 3G --live # add memory while running
# inside the machine, switch a new processor on:
echo 1 | sudo tee /sys/devices/system/cpu/cpu1/online
```

Automation

```
ansible all -m ping           # can I reach every machine?
ansible-playbook site.yml      # make the machines match the checklist
ansible-playbook site.yml --limit app # only the kitchen machine
```

The application

```

bash app/build.sh # build the WAR
curl http://192.168.122.12/labapp/health # through the waiter
curl http://192.168.122.11:8080/labapp/health # straight to the kitchen
(blocked)
ssh -i ~/.ssh/kvmlab ops@192.168.122.11 # log in to the kitchen

sudo journalctl -u tomcat10 -n 50 --no-pager # the kitchen's diary

```

Backups

```

sudo systemctl start tomcat-backup.service # make a backup now
sudo journalctl -u tomcat-backup.service -n 20 # did it verify?
systemctl list-timers tomcat-backup.timer # when does it run next?

sudo /usr/local/bin/tomcat-backup.sh verify <archive> # check one by hand
bash scripts/pull-backups.sh # copy them off the machine

bash scripts/backup-failure-drill.sh # prove the checker works

```

Monitoring

```

cd monitoring && docker compose up -d # start Prometheus + Grafana

docker compose logs --tail 20 # what are they saying?

curl -s localhost:9090/api/v1/targets # are the machines reporting?

curl -s localhost:3000/api/health # is Grafana alive?

```

Measuring and evidence

```

bash scripts/capacity-test.sh # the load test sweep
bash scripts/capture-evidence.sh # regenerate the evidence

ab -n 2000 -c 20 http://192.168.122.12/labapp/work # one load test by hand

```

The lab's two house rules

```

# 1. Before a session, keep a WSL session open so the machines are not
#    switched off between commands:
nohup wsl.exe -e bash -c "sleep 3600" >/dev/null 2>&1 &

# 2. After any WSL restart, check the network came back:
virsh net-list --all && ping -c1 192.168.122.1

```

Where you will meet all of this again

If the job says...	This project covers...
VMware / Hyper-V / KVM	Chapters 2 and 4: machines, snapshots, hot-add, the mapping table
Linux administration	Every chapter: systemd, users, permissions, logs, SSH
Ansible / configuration management	Chapter 4: roles, idempotency, and proof of it
Tomcat / WebLogic / Apache / nginx	Chapter 3: app servers, reverse proxies, virtual hosts
Shell scripting	The backup tool, the VM builder, the load test - all shell
Backup, DR, restore testing	Chapter 4 and the disaster recovery drill
Monitoring, Prometheus, Grafana	Chapter 5
Capacity planning	Chapter 5: the whole experiment
Networking, firewalls	Chapters 2 and 3
Containers, Docker	Chapter 5
Git, code review, CI	Chapter 4: branches, pull requests, clean history

Sentences worth being able to say out loud

Short versions you can say in an interview without rambling:

- **Idempotency:** “Running the playbook twice reports `changed=0` on the second run, which means every task inspected the machine and found nothing to do. To keep that true I had to make sure my build produced identical files, because a rebuild that changes bytes makes the deploy task report a change forever.”
- **Snapshot versus backup:** “A snapshot lives on the same disk as the machine, so it is for fast rollback. A backup is an independent, verified copy elsewhere, for disaster recovery. I proved the difference by destroying the machine - the snapshot died with the disk, and the backup brought the application back.”
- **Backup verification:** “I restore the archive into a scratch directory and compare checksums against a manifest from the source, so I am verifying the restore rather than trusting the archive. I also tamper with a manifest and confirm the checker refuses it, because a checker that only ever passes is worthless.”
- **Capacity:** “I load-tested through the proxy and read CPU back from Prometheus. The app VM was pinned at 100% at every size, the proxy sat at 14% and the host never passed 56%, so the app was the bottleneck and neither the proxy tier nor host exhaustion explained the flattening.”
- **Honesty about measurement:** “The same experiment run twice differed by about 20%, which is systematic rather than noise, so I quote the shape of the result rather than the exact numbers.”

- **Something that broke:** “My safety guard used `pgrep -x qemu-system-x86_64`, which can never match because Linux truncates process names to 15 characters. So the guard was always true and would have deleted the network bridge out from under running machines. It looked correct in review; only testing it against a live machine caught it.”

What to learn next, in a sensible order

1. **The same lab, automated one level higher.** Terraform can create the virtual machines themselves, instead of a shell script. Same idea as Ansible but for machines rather than settings inside machines.
2. **Automatic checks on every change.** GitHub Actions running `ansible-lint` and `yamllint` when you push. That is the beginning of CI.
3. **A second kitchen and a load balancer.** Add a second app machine and let Apache share orders between them, then switch one off and show that no requests fail. That is the beginning of high availability.
4. **HTTPS.** A self-signed certificate on Apache. It teaches how TLS actually works, and it is the first thing a security reviewer asks about.
5. **Containerise the application itself.** Build a Docker image that contains Tomcat and the WAR, so the app can be started anywhere in one command.
6. **Kubernetes.** The moment you have more than a couple of containers, people reach for it. It is a big subject; learn it after you have done steps 1 to 5.
7. **Something with a database.** Add a real database, and the backup conversation becomes genuinely interesting, because a database cannot just be copied while it is running.

The one-page summary

You built two computers inside your computer. One is a waiter that greets visitors, one is a kitchen that makes the food. You wrote a checklist so a program could build both machines identically, and proved it was safe to run twice. You took a photograph so you could undo mistakes in seconds, and you kept a copy of the recipe somewhere else so you could survive a disaster - then proved it by destroying the kitchen entirely and rebuilding it in under six minutes. You put meters on both machines, drew the numbers on a wall, and used them to find out exactly which part was the slow one. Then you locked the doors so visitors could only come through the front.

The part that will get you hired is not that it worked. It is that you can say what broke, why it broke, how you found out, and what you would do differently - because that is the job.